Chapter 4: Storing and Retrieving

Content

- Synchronization and Replication of Mobile Data
- Getting the Model Right.
- Android Storing and Retrieving Data.
- Working with a Content Provider.

Synchronization and Replication of Mobile Data

- Data synchronization is a method of establishing consistency among data from a data source to the target data storage and vice versa.
- In data synchronization, we have to keep multiple copies of a dataset in coherence with one another to maintain the data integrity.
- It provides continuous harmonization of the data over time.
- Data synchronization ensures accurate, secure & compliant data, and successful team & customer experiences.
- It assures congruence between each source of data and its different endpoints.
- As data comes in, it is cleaned, checked for errors, duplication, and consistency before being put to be used.
- Local synchronization involves devices and computers that are next to each other, while remote synchronization takes place over a mobile network.

Synchronization and Replication of Mobile Data

- Data must always be consistent throughout the data record.
- If data is modified in any way, changes must upgrade through every system in real-time to avoid mistakes, prevent privacy breaches, and ensure that the most up-to-date data is the only information available.
- Data synchronization ensures that all records are consistent, all the time.
- The following are the reasons why data synchronization is required in mobile computing:
  - Data synchronization is required between the mobile devices & their service providers to have proper communication.
  - It is also required among the devices, personal area computer, nearby wireless access points (Wi-Fi) & other nearby devices.
  - It is used to establish consistency in all devices.

Synchronization and Replication of Mobile Data

- The importance of data synchronization grows in step with increased accessibility to cloud-based data as well as access to mobile devices.
- Mobile devices use data for basic operation as well as personal information for apps, websites, and email.
- Data conflicts can result in errors and low data quality, which leads to a lack of trusted data.
- If data synchronization is properly implemented throughout a system, a business will see performance improvement in many areas, including:
  - Logistics and transportation
  - Sales team productivity
  - Order management
  - Invoice accuracy
  - Business systems
  - Cost efficiency & reputation management

Exploring Data Synchronization Methods

- File Synchronization: Faster & more error-proof than a manual copy technique.
  - This method is most used instead of home backups, external hard drives, or updating portable data via flash drive.
  - It ensures that two or more locations share the same data, and prevents duplication of files.
- Version Control: It aims to provide synchronized solutions for files that can be altered by more than one user at the same time.
- Distributed File Systems: When multiple file versions must be synced at the same time on different devices, those devices must always be connected for distributed file systems to work fine.
  - Few of these systems allow devices to disconnect for short periods of time, as long as data reconciliation is implemented before synchronization.
- Mirror Computing: Is used to provide different sources with an exact copy of a data-set.
  - Especially useful for backup, and it provides an exact copy to just one other location — source to target.

Getting the Model Right

- Every business wants to provide services to their customers, wherever they are and whatever they are doing.
- Some do this through their website, others use a mobile app.
- The challenge is that there is a multitude of different devices used by customers.
  1. So which devices should the app. be supported on?
  2. Can all be supported?
  3. Are there features on some devices that could be used to make the app. more engaging?
  4. Do we have the right skill-sets in-house to develop an app. that can be supported on all these different platforms?

...Cont'd

We have four models with their Pros and Cons

- Responsive Web App: mobile app. development devolves down to web app. development, but uses different techniques to produce a responsive web design that automatically adapts to the profile of the device that it is being displayed on.
- Web apps that are designed using responsive techniques make use of new HTML5 and CSS3 properties.
- Pros:
  - Create mobile apps. quickly and easily,
  - Cost of development will be kept low
- Cons:
  - Don't have access to underlying features of the device that it is running on, such as the GPS, camera...
  - User still left with a browser-based user experience.

...Cont'd

- Hybrid App: taking a web app. written in HTML5/JavaScript and wrap it in a native container app. such as Apache Cordova.
- Pros:
  - Removing the browser from the user experience.
  - Web app. code can be installed from an app store in exactly the same way as any other mobile apps.
  - Mobile app. development time remains shorter and predominantly makes use of readily available skills that may already be available in-house.
- Cons:
  - It may not be optimal for the device it is running on.
  - The UI elements rendered by the web app's CSS may not be representative or up-to-date with the native UI elements that a native app. would have → this could lead to an inconsistent UX across different devices or different versions of the OS (think iOS 6 vs. iOS 7).

...Cont'd

- Hybrid App II – Hybrid Mixed App: a more sophisticated approach is to take the hybrid approach, but infuse it with more platform-specific native code, to take advantage of more platform features.
- In this hybrid model, the app. logic is split between the web app code and the native container.
- **Pros**:
  - Tighter integration between the app. and the device hardware.
  - Improve the performance of some of the more intensive aspects of the application logic.
  - Hybrid approach remains viable as faster go-to market strategy for development.
- **Cons**:
  - Increased need for more platform-specific skills for each platform.
  - Retains the disadvantage of the earlier hybrid model and the responsive web app.

...Cont'd

- Native App: the application is written in a native language and environment for each platform, i.e. Objective-C on iOS for Apple, Java on Android, C sharp on Windows Phone, etc.
- Pros:
  - Best performance and provides full access to the underlying device hardware.
  - Performance of the mobile app. is often superior to its hybrid equivalent.
  - UI elements will be guaranteed to be consistent with the platform look and feel.
- Cons:
  - comes at a significant cost in terms of development

Android Storing and Retrieving Data.

- Android provides several options for you to save persistent application data.
- The solution you choose depends on your specific needs, such as whether
  - The data should be private to your application,
  - Accessible to other applications or
  - How much space your data requires.

...Cont'd:

- The main data storage options are the following:
  - Shared Preferences: Helps to store private primitive data in key-value pairs.
  - Internal Storage: Store private data on the device's own internal memory.
  - External Storage: Stores public data (anyone with USB cable can access) on the shared external storage.
  - SQLite Databases: Store structured data in the application's private database.
  - Network Connection: Store data on the web with your own network server.
  - Content provider: An optional component that exposes read/write access to your application data to whatever restrictions you want to impose.

...Cont'd: Shared Preferences

- The ***SharedPreferences*** class provides a general framework that allows you to save and retrieve persistent key-value pairs of primitive data types.
- You can use ***SharedPreferences*** to save any primitive data: ***boolean, float, int, long and string.***
- The data saved this way will persist across user sessions (even if your application is killed).
- **User Preferences**: Shared preferences are not strictly used for saving "**user preferences**" such as what ringtone a user has chosen.
- If you're interested in creating user preferences for your application;
  - Use **PreferenceActivity**, which provides an activity framework for you to create **user preferences**, that will be automatically persisted.

...Cont'd

- To get a **SharedPreferences** Object for your application, use one of two methods:
  - **getSharedPreferences()**: Use this method if you need multiple preferences files identified by name, which you specify with the first parameter.
  - **getPreferences()**: Use this method if you need only one preferences file for your activity.
    - **Because this will be the only preferences file for your Activity, you don't supply a name.**
  - *Context context = getActivity();*

  SharedPreferences sharedPref = context.getSharedPreferences(getString(R.string.preference_file_key), Context.MODE_PRIVATE);
  - *SharedPreferences sharedPref = getActivity().getPreferences(Context.MODE_PRIVATE);*
- **To write values:**
  - Call **edit()** → To get a **SharedPreferences.editor.**
  - Add values with methods such as → **putBoolean()** and **putString()**.
  - Commit the new values with → **commit()/apply()**
- **To read values**, use ***SharedPreferences*** methods such as
  - **getBoolean()** and
  - **getString()**

...Cont'd

```
public class _Shared_preferance extends Activity
{
    EditText txtInput;
    TextView textRead;
    Button btnSubmite, btnViewData;
    SharedPreferences sharedPreferances;
    public static final String keyFirstName = "nameKey";

    @Override
    protected void onCreate(Bundle savedInstanceState)
 {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_shared);
        textRead = findViewById(R.id.textView);
        txtInput = findViewById(R.id.txtResult);
        btnSubmite = findViewById(R.id.btnResultReturn);
        btnViewData = findViewById(R.id.btnView);
```

```
btnSubmite.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                putData();
            }
        });
btnViewData.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                viewData();
            }
        });
    }
  private void putData() {
        String firstName = txtInput.getText().toString();
        sharedPreferances= getSharedPreferences("DataStore",MODE_PRIVATE);
        SharedPreferences.Editor editor = sharedPreferances.edit();
        editor.putString(keyFirstName, firstName);
        editor.commit();
    }
  private void viewData(){
        if(sharedPreferances.contains(keyFirstName)){
            textRead.setText(sharedPreferances.getString(keyFirstName,""));
        }
    }
}
```

...Cont'd: Internal Storage

- You can save files directly on the device's internal storage.
- By default, files saved to the internal storage are private to your application and other applications cannot access them.
- When the user uninstalls the application, these files will be removed.
- To create and write a private file to internal storage;
  - Call **OpenFileOutput**() with the name of the file and operating mode
    - It will return FileOutputStream
  - Write to the file with write()
  - Close the stream with close()
    - e.g;
      - *String File_Name="Lab Three";*
      - *String string_In_File="Internal file";*
      - *FileOtputStream fos;*
      - *fos=OpenFileOutput(File_Name,Context.MODE PRIVATE);*
      - *fos.write(string.getBytes());*
      - *fos.close();*

...Cont'd

- **MODE PRIVATE**: creates a file in private mode or replace the already available file with the same name.
- To read a private file from internal storage;
  - Call **OpenFileInput()** with the name of the file to be read.
    - It will return FileInputStream
  - Read to the file with read()
  - Close the stream with close()
- **Saving Cache files:** Rather than storing data persistently, if you want to cache data, then you can use getCacheDir() to open a file that represents the internal folder where your application should save temporary cache files.
- If the device run out of storage it may delete by its own.
- When you remove the application such files will be removed.

...Cont'd

```
public class _InternalStorage extends Activity
 {
   private static final String FILE_NAME = "internalStorage.txt";
   Button btnView, btnSave;
   TextView txtView;
   EditText txtEdit;

   @Override
   protected void onCreate(Bundle savedInstanceState)
  {
      super.onCreate(savedInstanceState);
      setContentView(R.layout.activity_internal);

      txtEdit = findViewById(R.id.editText);
      txtView = findViewById(R.id.txtView);
      btnView = findViewById(R.id.btnView);
      btnSave = findViewById(R.id.btnsave);
```

...Cont'd

```
if (btnSave != null) {
        btnSave.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                String textData = txtEdit.getText().toString();
                try {
                    FileOutputStream fos = openFileOutput(FILE_NAME, MODE_PRIVATE);
                    OutputStreamWriter writer = new OutputStreamWriter(fos);
                    writer.write(textData);
                    writer.flush();
                    writer.close();
                    fos.close();
                    txtEdit.setText("");
                    Toast.makeText(getApplicationContext(), "file Saved to" + getFilesDir() + "/" +
FILE_NAME, Toast.LENGTH_LONG).show();
                } catch (IOException e) {
                    e.printStackTrace();
                }
            }
        });
    }
```

...Cont'd

```
if(btnView !=null) {
        btnView.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                try {
                    FileInputStream fis = openFileInput(FILE_NAME);
                    InputStreamReader isr = new InputStreamReader(fis);
                    BufferedReader br = new BufferedReader(isr);
                    String text;
                   StringBuilder sb = new StringBuilder();
                    while ((text = br.readLine()) != null) {
                        sb.append(text).append("\n");
                    }
                    br.close();
                    isr.close();
                    fis.close();
                    txtView.setText(sb.toString());
                } catch (FileNotFoundException e) {
                    txtView.setText("File not found: " + FILE_NAME);
                } catch (IOException e) {
                    txtView.setText("Error reading file: " + e.getMessage());
                }
            }
        });
    }
}
```

...Cont'd: External Storage

- Every android compatible device supports external storage for saving files.
- The storage may be;
  - Removable media such as SD card or
  - Non-removable
- External storage files are world-readable in that they can be modified by the user when USB mass storage to transfer files is enabled.
- To check if the external media is available you have to use the method → **getExternalStorageState()**.
- Other methods include;
  - **getFilesDir**() → to get the absolute path of the file.
  - **getDir**() → create or open an existing directory.
  - **deleteFile** → deletes a file from the storage.
  - **fileList** → returns array of files.

...Cont'd

- To use External Storage: you should provide access permission to write and read to the external storage in the manifest file.

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    package="com.example.externalstorage">
    <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>;
    <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>;
```

```
public class _ExternalStorage extends Activity
 {
    private static final String FILE_NAME = "externalFile.txt";
    String path = "File";
    Button btnView, btnSave;
    TextView txtView;
    EditText txtEdit;
    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_external);
        txtView = findViewById(R.id.txtView);
        txtEdit = findViewById(R.id.editText);
        btnView = findViewById(R.id.btnView);
        btnSave = findViewById(R.id.btnsave);
        btnSave.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                putData();
            }
        });
```

```
btnView.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View view) {
            ViewData();
        }
    });
  }
public void putData() {
      String fileContent = txtEdit.getText().toString().trim();
      if(!isExternalStorageAvailable())
          {
              txtEdit.setEnabled(false);
          }
      if (!fileContent.equals("")) {
          File externalFile = new File(getExternalFilesDir(path), FILE_NAME);
          FileOutputStream fos = null;
          try {
              fos = new FileOutputStream(externalFile);
              fos.write(fileContent.getBytes());
          } catch (IOException e) {
              e.printStackTrace();
          }
          txtEdit.setText("");
          if (isExternalStorageAvailable()) {
              Toast.makeText(getApplicationContext(), "File Saved to external Storage",
Toast.LENGTH_LONG).show();
          } else {
              Toast.makeText(getApplicationContext(), "External storage is not available",
Toast.LENGTH_LONG).show();
          }
      }
  }
```

```
private boolean isExternalStorageAvailable() {
        String extStorage = Environment.getExternalStorageState();
        return Environment.MEDIA_MOUNTED.equals(extStorage);
    }
 private void ViewData() {
        File externalFile = new File(getExternalFilesDir(path), FILE_NAME);
        StringBuilder stringBuilder = new StringBuilder();
        try (FileReader filereader = new FileReader(externalFile);
             BufferedReader bufferedReader = new BufferedReader(filereader)) {
            String str = bufferedReader.readLine();
            while (str != null) {
                stringBuilder.append(str).append("\n");
                str = bufferedReader.readLine();
            }
            txtView.setText(stringBuilder.toString());
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

...Cont'd: SQLite:

- **SQLite:** Is a relational database technology that's used most often when the developer requires an **embedded database.**
- It is included with the android system and can be easily used in your android app.
- **Creating a Database in Android**: In order to write data to a database, we need to create our database first.
- To create a database in **SQLite in Android**, We need to create a subclass of the class ***SQLiteOpenHelper***.
- This class provides the functionality to create a database and calls a function that can be overridden by the derived class.
- **SQLiteOpenHelper** Constructor accepts **four** arguments, which are
  - **Context**: This is the context that will be used to create the database.
  - **Name**: The name of the new database file.
  - **Factory**: A cursor factory (you can usually pass this as null)
  - **Version**: The version of the database.
    - This number (version number) is used to identify if there is an upgrade or downgrade of the database.

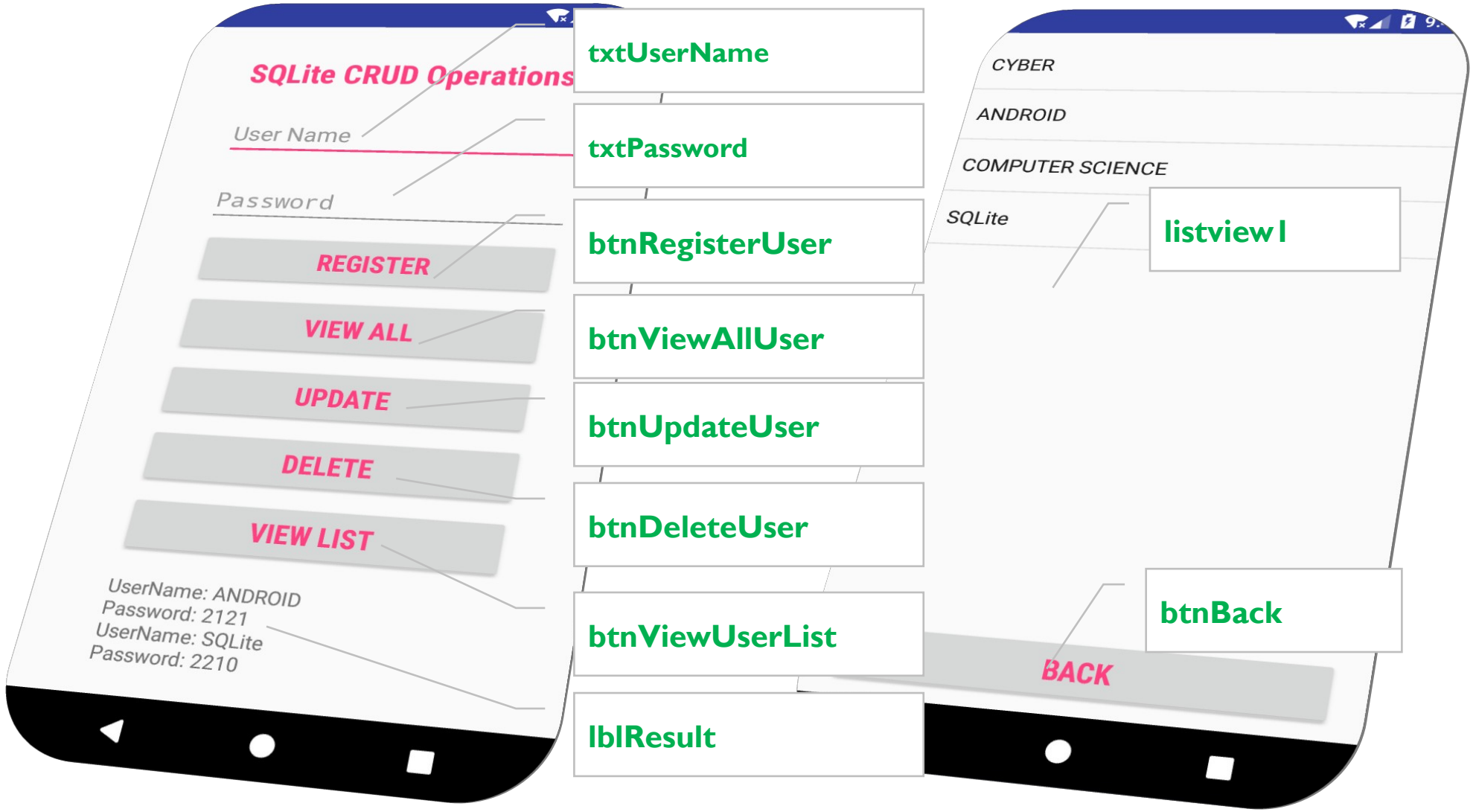
...Cont'd

- In the constructor that we create, we pass the name of the database that we want to create.
- We also override the following methods:
- onCreate: This method is called when we create the database.
  - It is passed the SQLiteDatabase reference, which we can use to perform various operations on the database.
  - In this method, We use the function execSQL to execute a SQL query to create a table we want.
- onUpgrade: This method is called whenever the database is upgraded.
  - In this method, SQLiteDatabase, the oldVersion number and newVersion number are passed.
  - In this function, we simply drop the table if it already exists and create a new one.

...Cont'd

- To write data to database → use **getWritableDatabase()**
- To read data from database → use **getReadableDatabase()**
- **Adding Values within the Database**:
  - Once the database is created, now we will add values to the database.
  - We use the insert function on the **SQLiteDatabase** class.
  - The insert function takes three arguments: The name of the table in which to insert values, the name of one of the column of the table, and a reference to a ContentValues.
- **Running a raw SQL query**:
  - Each of the SQLite quires will return cursor which will point to all the rows found by your query.
  - Using cursor is the main mechanism to navigate results from the database query and read rows & columns.

Step 1: UI Design



Step 2: DatabaseHelper.java

```
public class DatabaseHelper extends SQLiteOpenHelper {
    public static final String DATABASE_NAME = "Student.db";
    public static final String TABLE_NAME = "tblStudent";

    public static final String COLUMN_1 = "userName";
    public static final String COLUMN_2 = "password";

    public DatabaseHelper(Context context){
        super(context, DATABASE_NAME, null,1);
    }

    public void onCreate(SQLiteDatabase db){
        db.execSQL("CREATE TABLE "+TABLE_NAME+
        "(userName TEXT PRIMARY KEY, password TEXT)");
    }

    public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVesrion){
        db.execSQL("DROP TABLE IF EXISTS "+TABLE_NAME);
    }
```

Database Version

Insert Record

```
public boolean registerUser(String userName, String password){

    SQLiteDatabase db = this.getWritableDatabase();

    ContentValues contentValues = new ContentValues();

    contentValues.put(COLUMN_1, userName);
    contentValues.put(COLUMN_2, password);

    long result = db.insert(TABLE_NAME, null, contentValues);
    db.close();

    if(result==-1) {
        return  false;
    }else {
        return  true;
    }
}
```

Retrieve Record

```
public Cursor getAllUser(){ //Retrieve Record, display it to
TextView

     SQLiteDatabase db =this.getWritableDatabase();

    Cursor result = db.rawQuery("SELECT *FROM "+TABLE_NAME, null);

     return result;
}
```

Update Record

```
public boolean updateUser(String userName, String password) {

        SQLiteDatabase db = this.getWritableDatabase();

        ContentValues contentValues = new ContentValues();
        contentValues.put(COLUMN_1, userName);
        contentValues.put(COLUMN_2, password);

        int result = db.update(TABLE_NAME, contentValues,
       "userName=?", new String[]{userName});

        if (result > 0) {
            return  true;
        }
        else {
            return false;
        }
}
```

Delete Record

```
public Integer deleteUser(String userName){ //Delete Record

        SQLiteDatabase db = this.getWritableDatabase();

        int result = db.delete(TABLE_NAME,"userName=?",
        new String[]{userName});

        return result;
}
```

View User List – (on ListView)

```
public Cursor  viewUserList() { //Displaying on ListView

    SQLiteDatabase db = this.getReadableDatabase();

    String[] columns = new String[]{COLUMN_1, COLUMN_2};

    Cursor cursor = db.query(TABLE_NAME, columns,
    null, null, null, null, null);

    return cursor;
}
```

Step 3: MainActivity.java

```
public class MainActivity extends AppCompatActivity {
    EditText txtUserName, txtPassword;
    TextView txtResult;
    Button btnRegisterUser, btnViewAllList, btnUpdateUser, btnDeleteUser, btnViewUserList;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        DatabaseHelper db = new DatabaseHelper(this);
        txtUserName = (EditText) findViewById(R.id.txtUserName);
        txtPassword = (EditText) findViewById(R.id.txtPassword);
        txtResult = (TextView) findViewById(R.id.txtResult);
        btnRegisterUser = (Button) findViewById(R.id.btnRegisterUser);
        btnViewAllList = (Button) findViewById(R.id.btnViewAllList);
        btnUpdateUser = (Button) findViewById(R.id.btnUpdateUser);
        btnDeleteUser = (Button) findViewById(R.id.btnDeleteUser);
       btnViewUserList = (Button) findViewById(R.id.btnViewUserList);
```

Register Button Action

```
btnRegisterUser.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        String userName = txtUserName.getText().toString();
        String password = txtPassword.getText().toString();

        boolean result = db.registerUser(userName, password);

        if(result==true){
            Toast.makeText(getApplicationContext(),
            "Registered!", Toast.LENGTH_LONG).show();
        }
        else {
            Toast.makeText(getApplicationContext(),
            "Failed to Register!", Toast.LENGTH_LONG).show();
        }
    }
});
```

ViewAllUser Button Action(Display on TextView)

```
btnViewAllUser.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
       Cursor result = db.getAllUser();

        StringBuffer stringBuffer = new StringBuffer();
        if(result!=null && result.getCount()>0) {
            while (result.moveToNext()) {
              stringBuffer.append("UserName: " + result.getString(0) + "\n");
              stringBuffer.append("Password: " + result.getString(1) + "\n");
            }
            txtResult.setText(stringBuffer.toString());
        }
        else {
             Toast.makeText(getApplicationContext(), "No Data to Retrieve!",
             Toast.LENGTH_LONG).show();
         }
    }
});
```

Update Button Action

```
btnUpdateUser.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        String userName = txtUserName.getText().toString();
        String password = txtPassword.getText().toString();

        boolean result = db.updateUser(userName, password);

        if(result==true){
            Toast.makeText(getApplicationContext(),
            "Data Updated!", Toast.LENGTH_LONG).show();
        }
        else {
            Toast.makeText(getApplicationContext(),
            "No Data to Update!", Toast.LENGTH_LONG).show();
        }
    }
});
```

Delete Button Action

```
btnDeleteUser.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
     String userName = txtUserName.getText().toString();
     int result = db.deleteUser(userName);

     if(result==1){
           Toast.makeText(getApplicationContext(),
           "Data Deleted!", Toast.LENGTH_LONG).show();
     }
     else {
       Toast.makeText(getApplicationContext(),
       "Data Not Deleted!", Toast.LENGTH_LONG).show();
     }
     }
   });
```

View User List Button Action

```
btnViewUserList.setOnClickListener(new View.OnClickListener() {
//Displaying to List View

    @Override
    public void onClick(View v) {

        Intent i = new Intent(getApplicationContext(),
        UserList.class);

        startActivity(i);
    }
  });
 }
}
```

View List (on ListView) – UserList.java

```
public class UserList extends Activity { //Displaying to List View
    ArrayList<String> users = new ArrayList<String>();
    ArrayAdapter<String> adapter;
    ListView listView;
    Button btnBack;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_user_list);

        DatabaseHelper db = new DatabaseHelper(this);

        adapter = new ArrayAdapter<String>(this,
        android.R.layout.simple_list_item_1, users);

        btnBack = (Button) findViewById(R.id.btnBack);
        listView = (ListView)findViewById(R.id.listview1);
```

...Cont'd

```
Cursor cursor = db.viewUserList();
if(cursor!=null && cursor.getCount()>0) {
    while (cursor.moveToNext()) {
        String userName = cursor.getString(0);
        users.add(userName);
    }
    db.close();
    listView.setAdapter(adapter);
} else {
    Toast.makeText(getApplicationContext(),"No Data Found!",
    Toast.LENGTH_LONG).show();
}
listView.setOnItemClickListener(new AdapterView.OnItemClickListener() {
    @Override
    public void onItemClick(AdapterView<?> parent, View view, int
    position, long id) {
        Toast.makeText(getApplicationContext(), users.get(position),
        Toast.LENGTH_SHORT).show();
    }
});
```

Back Button Action

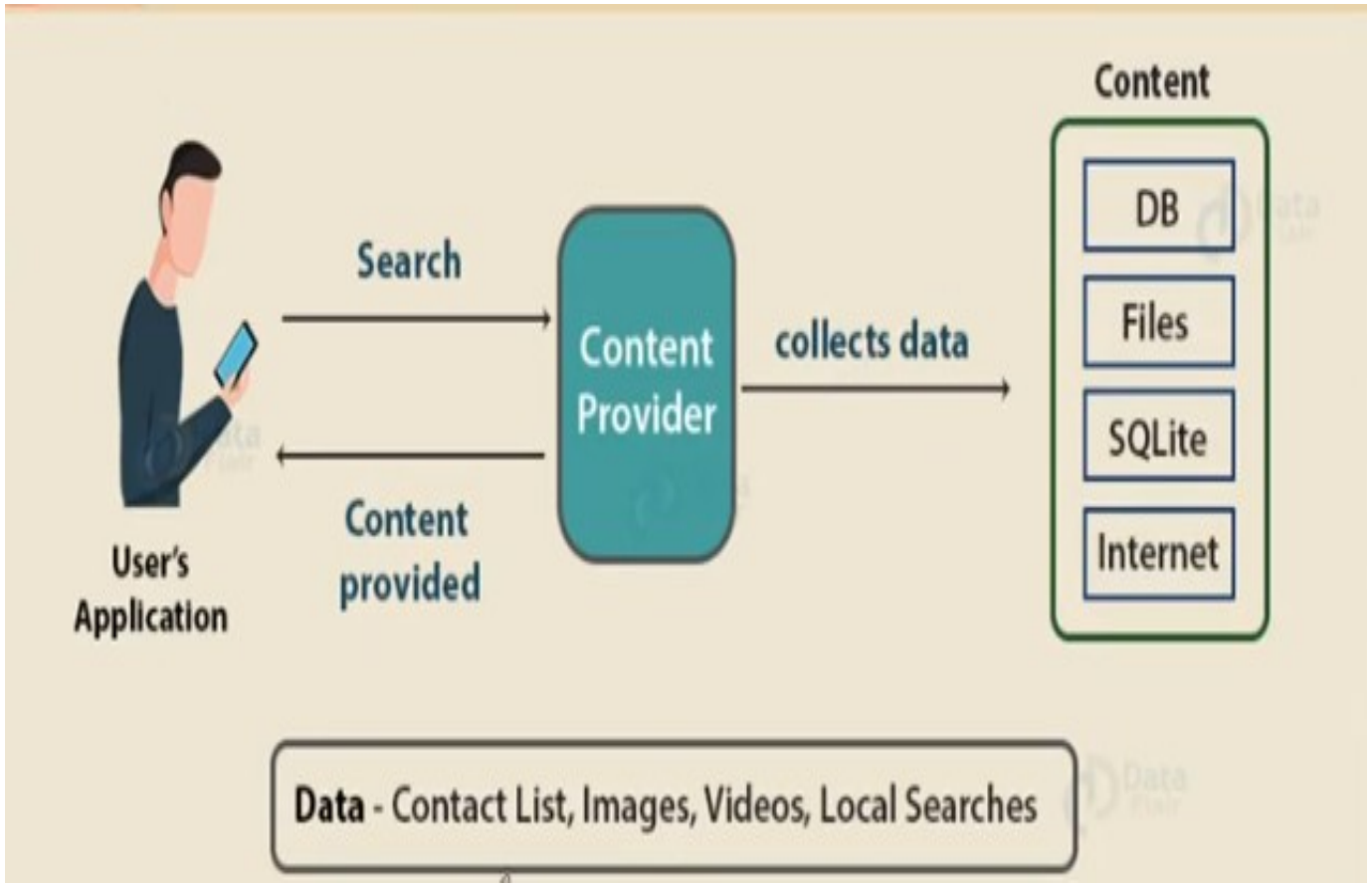
```
btnBack.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {

        Intent in = new Intent(getApplicationContext(),
        MainActivity.class);
        startActivity(in);
    });
   }
}
```

Working with ContentProvider's

- In Android, ContentProvider's are very important components that serves the purpose of a relational database to store the data of applications.
- The role of the ContentProvider's in the android system is like a **central repository** in which data of the applications are stored, and it also facilitates other applications to securely access and modify that data based on the user requirements.
- Android system allows the contentProvider to store the application data in several ways.
- Users can manage to store the application data like images, audio, videos, and personal contact information by storing them in SQLite database, in files, or even on a network.
- In order to share the data, content providers have certain permissions that are used to grant or restrict the rights to other applications to interfere with the data.

...Cont'd


Content
DB
Files
SQLite
Internet
Search
Content Provider
collects data
Content provided
User's Application
Data - Contact List, Images, Videos, Local Searches

...Cont'd

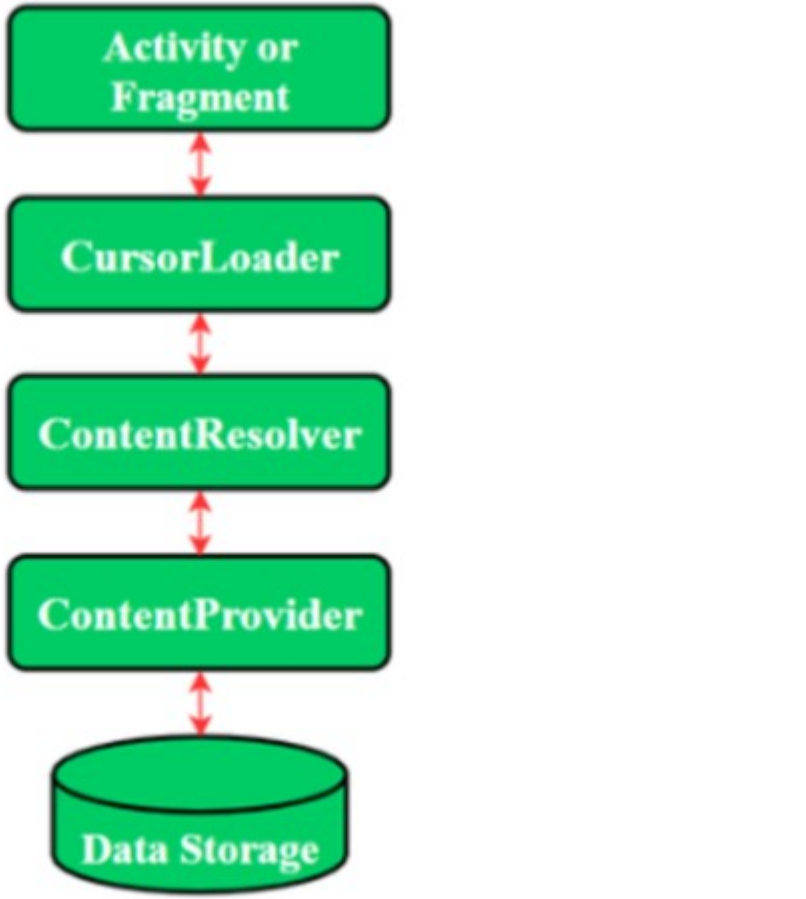
- Content URI (Uniform Resource Identifier): Is the key concept of content providers.
- To access the data from a content provider, URI is used as a query string.
- Details of different parts of Content URI -> **URI:** content://authority/optionalPath/optionalID
  - Content://: Mandatory part of the URI as it represents that the given URI is a Content URI.
  - Authority: Signifies the name of the content provider like contacts, browser, etc. This part must be unique for every content provider.
  - OptionalPath: Specifies the type of data provided by the content provider.
    - It is essential as this part helps content providers to support different types of data that are not related to each other like audio and video files.
  - OptionalID: It is a numeric value that is used when there is a need to access a particular record.
  - If an ID is mentioned in a URI then it is an id-based URI otherwise a directory-based URI.
  - E.g.
    - Content://media/external/images/media/9134
    - Content://com.android.contacts/data/123

...Cont'd

- Operations in Content Provider:
- Four fundamental operations are possible in content provider namely **Create, Read, Update, and Delete.**
- These operations are often termed as CRUD operations.
  - **Create**: Operation to create data in a content provider.
  - **Read**: Used to fetch data from a content provider.
  - **Update**: Modify existing data.
  - **Delete**: Remove existing data from the storage.
- Working of the content provider UI components of android applications like Activity and Fragments use an object **CursorLoader** to send query requests to **ContentResolver**.
- The **ContentResolver** object sends requests (like create, read, update & delete) to the content provider as a client.
- After receiving a request, content provider processes it and returns the desired result.

...Cont'd

- Below is a diagram to represent these processes in pictorial form.



...Cont'd

- **Creating a content provider**
  - Following are the steps which are essential to follow in order to create a content provider:
  - Create a class in the same directory where the MainActivity file resides and this class must extend the ContentProvider base class.
  - To access the content, define a content provider URI address.
  - Create a database to store the application data.
  - Implement the six abstract methods of ContentProvider class.
  - Register the content provider in AndroidManifest.xml file using provider tag.

...Cont'd

- The following are the six abstract methods and their description which are essential to override as the part of content provider class:
  - query(): a method that accepts arguments and fetches the data from the desired table.
    - Data is retrieved as a cursor object.
  - insert(): is used to insert a new row in the database of the content provider.
    - It returns the content URI of the inserted row.
  - update(): is used to update the fields of an existing row.
    - It returns the number of rows updated.
  - delete(): is used to delete the existing rows.
    - It returns the number of rows deleted.
  - getType(): returns the multipurpose Internet mail extension (MIME) type of data to the given Content URI.
  - onCreate(): as the content provider is created, the android system calls this method immediately to initialize the provider.

Example: Content Providers

...Cont'd

- The content of activity_main.xml

```
<ImageButton
     android:layout_width="wrap_content“
     android:layout_height="wrap_content“
     android:id="@+id/imageButton"
     android:src="@drawable/logo“
     android:layout_centerHorizontal="true" />

<Button
     android:layout_width="wrap_content“
     android:layout_height="wrap_content“
     android:id="@+id/button2“
     android:text="Add Name“
     android:onClick="onClickAddName"/>

<EditText
     android:layout_width="wrap_content“
     android:layout_height="wrap_content“
     android:id="@+id/editText“ />
```

```
<EditText
     android:layout_width="wrap_content“
     android:layout_height="wrap_content“
     android:id="@+id/editText2“
     android:hint="Name“
     android:textColorHint="@android:color/holo_blue_light" />

<EditText
     android:layout_width="wrap_content“
     android:layout_height="wrap_content“
     android:id="@+id/editText3“
     android:hint="Grade"
     android:textColorHint="@android:color/holo_blue_bright" />

<Button
     android:layout_width="wrap_content"
     android:layout_height="wrap_content“
     android:text="Retrieve Student“
     android:id="@+id/button“
     android:onClick="onClickRetrieveStudents"/>
```

…Cont'd

- **The content of MainActivity.java**

```
public class MainActivity extends Activity {
     @Override
     protected void onCreate(Bundle savedInstanceState) {
          super.onCreate(savedInstanceState);
          setContentView(R.layout.activity_main);
     }
     public void onClickAddName(View view) {
          // Add a new student record
          ContentValues values = new ContentValues();
          values.put(StudentsProvider.NAME, ((EditText)findViewById(R.id.editText2)).getText().toString());
          values.put(StudentsProvider.GRADE, ((EditText)findViewById(R.id.editText3)).getText().toString());
          Uri uri = getContentResolver().insert(StudentsProvider.CONTENT_URI, values);
          Toast.makeText(getBaseContext(), uri.toString(), Toast.LENGTH_LONG).show();
     }
     public void onClickRetrieveStudents(View view) {
          // Retrieve student records
          String URL = "content://com.example.MyApplication.StudentsProvider";
          Uri students = Uri.parse(URL);
          Cursor c = managedQuery(students, null, null, null, "name");
          if (c.moveToFirst()) {
          do{
               Toast.makeText(this, c.getString(c.getColumnIndex(StudentsProvider._ID)) + ", " +
               c.getString(c.getColumnIndex( StudentsProvider.NAME)) + ", " +
               c.getString(c.getColumnIndex( StudentsProvider.GRADE)), Toast.LENGTH_SHORT).show();
          } while (c.moveToNext());
          }
     }
}
```

```
public class StudentsProvider extends ContentProvider {
     static final String PROVIDER_NAME = "com.example.MyApplication.StudentsProvider";
     static final String URL = "content://" + PROVIDER_NAME + "/students";
     static final Uri CONTENT_URI = Uri.parse(URL);
     static final String _ID = "_id";
     static final String NAME = "name";
     static final String GRADE = "grade";
     private static HashMap<String, String> STUDENTS_PROJECTION_MAP;
     static final int STUDENTS = 1;
     static final int STUDENT_ID = 2;
     static final UriMatcher uriMatcher;
     Static {
          uriMatcher = new UriMatcher(UriMatcher.NO_MATCH);
          uriMatcher.addURI(PROVIDER_NAME, "students", STUDENTS);
          uriMatcher.addURI(PROVIDER_NAME, "students/#", STUDENT_ID);
     }
      /** * Database specific constant declarations */
     private SQLiteDatabase db;
     static final String DATABASE_NAME = "College";
     static final String STUDENTS_TABLE_NAME = "students";
     static final int DATABASE_VERSION = 1;
     static final String CREATE_DB_TABLE = " CREATE TABLE " + STUDENTS_TABLE_NAME +
     " (_id INTEGER PRIMARY KEY AUTOINCREMENT, " + " name TEXT NOT NULL, " + " grade TEXT NOT NULL);";
      /** * Helper class that actually creates and manages * the provider's underlying data repository. */
     private static class DatabaseHelper extends SQLiteOpenHelper {
          DatabaseHelper(Context context){
          super(context, DATABASE_NAME, null, DATABASE_VERSION);
     }
```

...Cont'd

```
@Override
public void onCreate(SQLiteDatabase db) {
     db.execSQL(CREATE_DB_TABLE);
}
@Override
public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
     db.execSQL("DROP TABLE IF EXISTS " + STUDENTS_TABLE_NAME); onCreate(db);
}
}
@Override
public boolean onCreate() {
     Context context = getContext();
     DatabaseHelper dbHelper = new DatabaseHelper(context);
     /** * Create a write able database which will trigger its
          * creation if it doesn't already exist. */
     db = dbHelper.getWritableDatabase();
     return (db == null)? false:true;
}
@Override
public Uri insert(Uri uri, ContentValues values) {
     /** * Add a new student record */
     long rowID = db.insert(STUDENTS_TABLE_NAME, "", values);
     /** * If record is added successfully */
     if (rowID > 0) {
          Uri _uri = ContentUris.withAppendedId(CONTENT_URI, rowID);
          getContext().getContentResolver().notifyChange(_uri, null); return _uri;
     }
     throw new SQLException("Failed to add a record into " + uri);
}
```

...Cont'd

```
@Override
public Cursor query(Uri uri, String[] projection, String selection, String[] selectionArgs, String
sortOrder) {

     SQLiteQueryBuilder qb = new SQLiteQueryBuilder();
     qb.setTables(STUDENTS_TABLE_NAME);

     switch (uriMatcher.match(uri)) {
          case STUDENTS:
               qb.setProjectionMap(STUDENTS_PROJECTION_MAP);
          break;

          case STUDENT_ID:
               qb.appendWhere( _ID + "=" + uri.getPathSegments().get(1));

          break;

          default:
          }
          if (sortOrder == null || sortOrder == ""){
               /** * By default sort on student names */
               sortOrder = NAME;
          }
          Cursor c = qb.query(db, projection, selection, selectionArgs, null, null, sortOrder);

          /** * register to watch a content URI for changes */
          c.setNotificationUri(getContext().getContentResolver(), uri);
          return c;
 }
```

...Cont'd

```
@Override
public int delete(Uri uri, String selection, String[] selectionArgs) {
     int count = 0;

     switch (uriMatcher.match(uri)){
          case STUDENTS: count = db.delete(STUDENTS_TABLE_NAME, selection, selectionArgs);
          break;

          case STUDENT_ID:
               String id = uri.getPathSegments().get(1);
               count = db.delete( STUDENTS_TABLE_NAME, _ID + " = " + id + (!TextUtils.isEmpty(selection) ? "
               AND (" + selection + ')' : ""), selectionArgs);
          break;

          default:
               throw new IllegalArgumentException("Unknown URI " + uri);
     }

     getContext().getContentResolver().notifyChange(uri, null);
     return count;
}
```

...Cont'd

```
    @Override
    public int update(Uri uri, ContentValues values, String selection, String[] selectionArgs) {
        int count = 0;
        switch (uriMatcher.match(uri)) {
            case STUDENTS: count = db.update(STUDENTS_TABLE_NAME, values, selection, selectionArgs);
            break; case STUDENT_ID: count = db.update(STUDENTS_TABLE_NAME, values, _ID + " = " +
            uri.getPathSegments().get(1) + (!TextUtils.isEmpty(selection) ? " AND (" +selection + ')' : ""),
            selectionArgs);
            break;
            default: throw new IllegalArgumentException("Unknown URI " + uri );
        }
        getContext().getContentResolver().notifyChange(uri, null);
        return count;
    }
    @Override
    public String getType(Uri uri) {
        switch (uriMatcher.match(uri)){
            /** * Get all student records */
            case STUDENTS: return "vnd.android.cursor.dir/vnd.example.students";
            /** * Get a particular student */
            case STUDENT_ID: return "vnd.android.cursor.item/vnd.example.students";
            default: throw new IllegalArgumentException("Unsupported URI: " + uri);
        }
    }
}
```

The End!